hashlock.



Security Audit

Line Next 2nd (Smart Wallet)

October 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Conclusion	32
Our Methodology	34
Disclaimers	36
About Hashlock	37

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Line Next team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

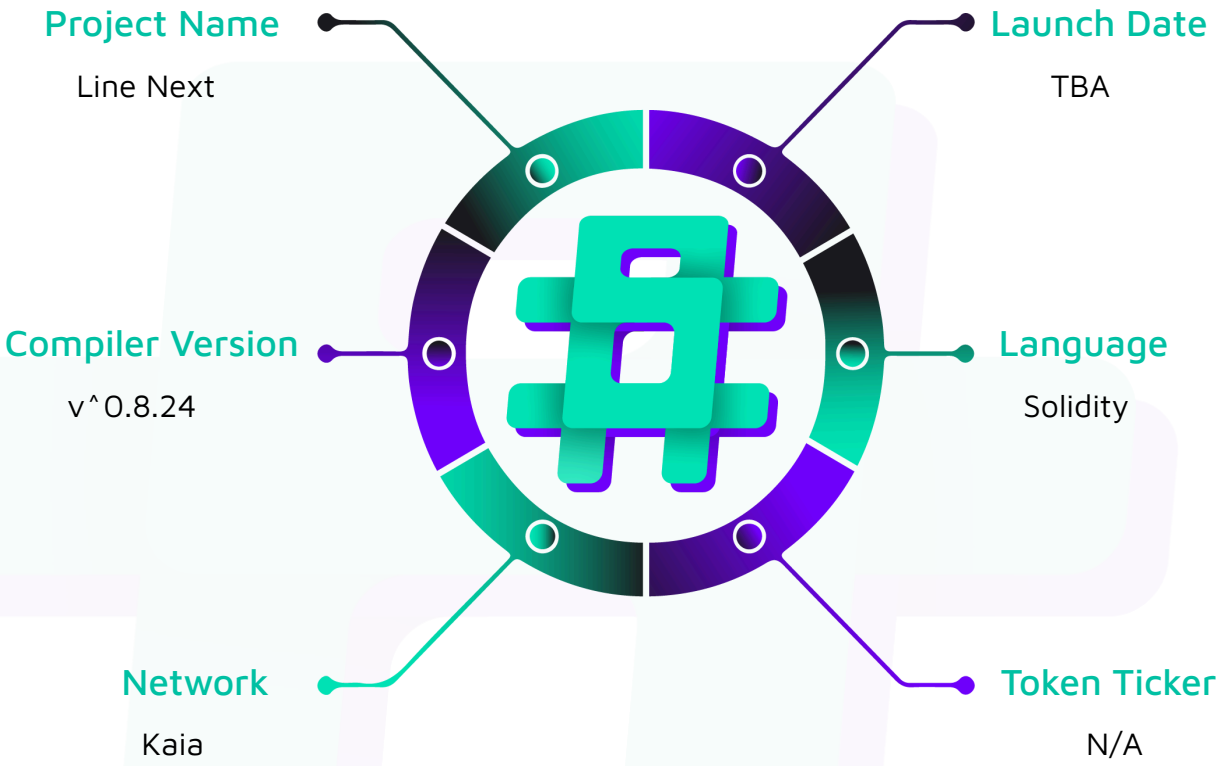
Line Next is an EIP-7702 smart-account (account abstraction) system that lets an EOA delegate execution to a recovery-enabled implementation. Users register a WebAuthn / passkey credential (secp256r1) against their account, and can recover control of the account's assets through a combination of WebAuthn signature verification and a guardian multisig (social recovery).

In short, the account owner is not dependent on a single private key: if the EOA key is lost, a quorum of guardians plus the registered passkey can authorize recovery transactions, while a separate governance contract manages the guardian set, quorum, and executor.

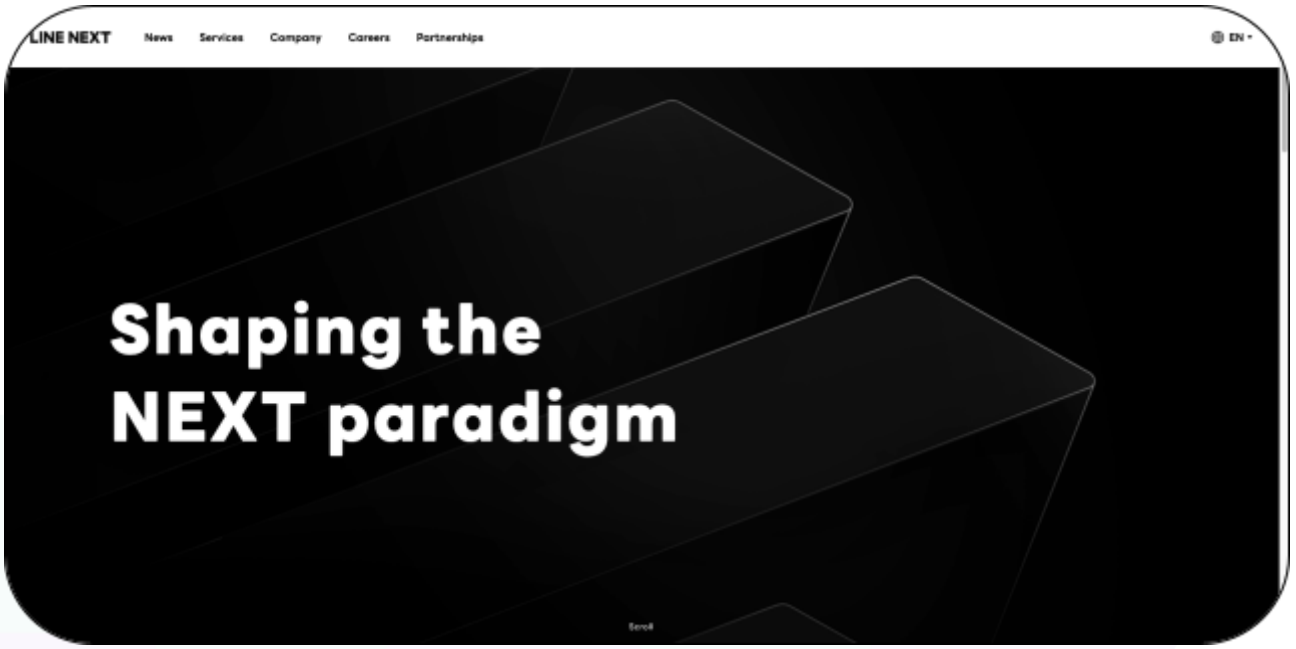
Project Name: Line Next
Project Type: Smart Wallet
Compiler Version: ^0.8.24
Website: www.linenextcorp.com
Logo:

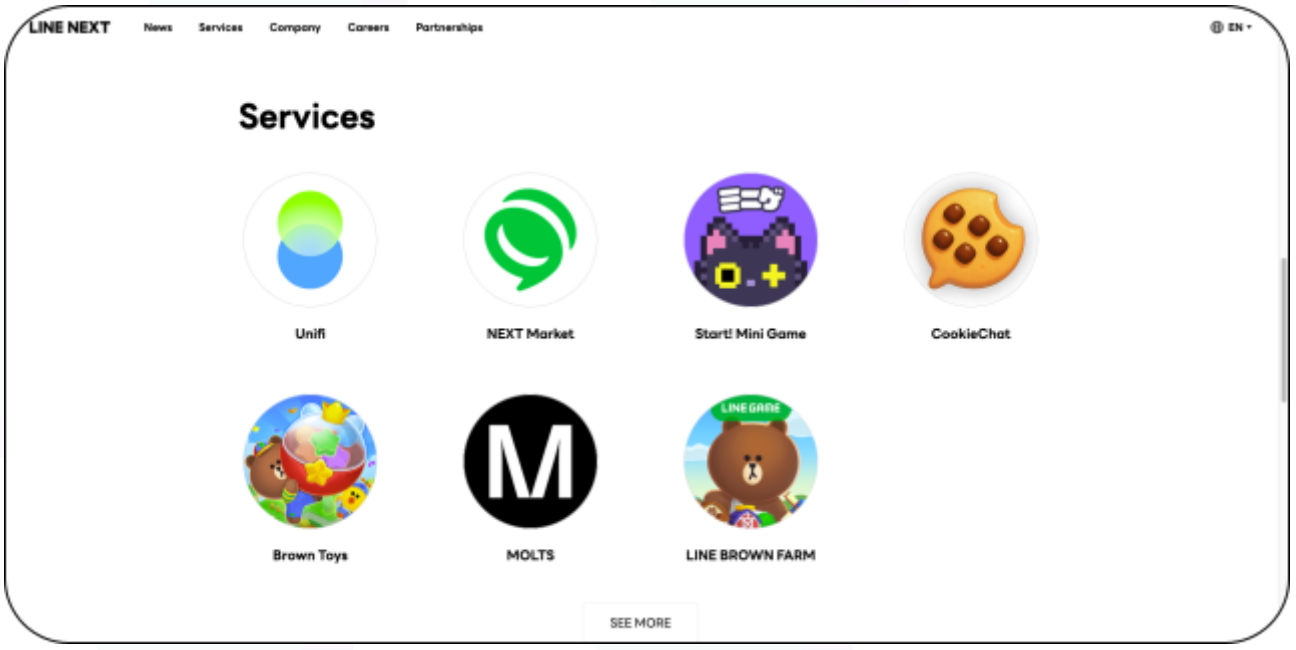


Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Line Next project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Line Next Smart Contracts**
Network	**Kaia**
Language	**Solidity**
Audit Date	**October, 2026**
Contract 1	Delegate7702Recovery.sol
Contract 2	Delegate7702Verifier.sol
Contract 3	IDelegate7702Verifier.sol
Contract 4	VaultLib.sol
Contract 5	VaultV2.sol
Contract 6	MigrationLib.sol
Contract 7	PrimeLib.sol
Contract 8	WithdrawalQueue.sol
Contract 9	RolesLib.sol
Contract 10	VaultRolesV2.sol
Contract 11	PrimeQueue.sol
Contract 12	BalanceLib.sol
Audited GitHub Commit Hash	eafe977169776cd5a16d2fc589e562ab49977e6c
Fix Review GitHub Commit Hash	0c1e498542b5cb88096d519d77e6c11a7c9da27b

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

5 Medium severity vulnerabilities

2 Low severity vulnerability

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope involves the smart contracts of the Line Next project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Line Next project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] contracts/vault/v2/MigrationLib.sol#acceptMigrationRequestImpl - Source migration sends liquid assets before the guardian delay

Description

`acceptMigrationRequest()` never reads `eta`. It checks that the caller is `proposedVault` and that the state is still `Proposed`, then marks the migration accepted and transfers the liquid balance. The 24-hour check exists only on the destination, inside `acceptVaultFromImpl`. A nominated contract doesn't have to use that function.

Vulnerability Details

`proposeVault` is `onlyManager` and stores a 24-hour delay. Cancellation is guardian-only, so a stolen manager key cannot cancel a migration on its own:

```
m.eta = uint32(block.timestamp + _MIGRATION_TIMELOCK_VALUE);
```

`acceptVaultFromImpl` waits for that `eta`. The surrounding note says `oldVault.acceptMigrationRequest()` performs the same check:

```
if (block.timestamp < abi.decode(data, (uint32))) revert
VaultMigrationTimelockNotReady();
```

The source function does not. It accepts the caller and the state, then transfers:

```
if (msg.sender != proposedVault_) revert NotMigrationSource();
if (m.state != _MIGRATION_PROPOSED) revert MigrationStateInvalid();

m.state = _MIGRATION_ACCEPTED;
m.accepted = true;

amount = actualBalance < expectedBal ? actualBalance : expectedBal;
if (amount > 0) {
    BalanceLib.decreaseExpectedBalance(amount);
```

```
    IERC20(asset_).safeTransfer(proposedVault_, amount);
}
```

`proposeVault` only checks that the destination answers a set of view calls. A manager key can nominate a contract that passes those calls. That contract then calls `acceptMigrationRequest()` immediately. The manager address itself is not the allowed caller; the nominated contract is. After acceptance, `migrationAccepted` is true, so `cancelVaultProposal` reverts.

Impact

The guardian delay currently doesn't protect the source vault. Once a stolen manager key nominates its own contract, that contract can take the liquid balance, `min(token balance, expectedBalance)`, without waiting 24 hours. The guardian has no ability cancel afterwards.

Recommendation

Require `block.timestamp >= eta` inside `acceptMigrationRequestImpl` before changing state or transferring assets.

Status

Resolved

Medium

[M-01] contracts/strategies/SuperEarnStrategy.sol#claimWithdrawal - One redemption can spend another request's proceeds

Description

Both claim paths pay from the strategy's whole token balance, capped only by that request's recorded size. `req.assets` is the preview at request time, not the amount that later arrived for that request. Several unclaimed CooldownVault redemptions share one balance.

Vulnerability Details

The auto-claim path transfers `min(entire balance, req.assets)`:

```
uint256 autoClaimedBalance = assetToken.balanceOf(address(this));
if (autoClaimedBalance > 0) {
    uint256 autoClaimedTransfer = autoClaimedBalance < req.assets ?
autoClaimedBalance : req.assets;
    assetToken.safeTransfer(unifiVault, autoClaimedTransfer);
}
```

`emergencyClaimWithdrawal` does the same when the CooldownVault request is already claimed:

```
uint256 balance = assetToken.balanceOf(address(this));
transferAmount = balance < req.assets ? balance : req.assets;
```

If request A's preview is higher than the tokens that actually arrived for A, A's claim can include tokens that arrived for B. `claimRedemptions` is permissionless and removes a pending redemption only when `claimWithdrawal` returns. If A takes all of B's tokens, B is already claimed in CooldownVault, the strategy balance is 0, and B's later claim reverts `InsufficientClaimedAssets`. The revert rolls B's local claimed flag back, so B stays pending and every retry fails.

Impact

One redemption is cleared using another request's proceeds. The second request can remain pending forever.

Recommendation

Attribute proceeds per request. Record the balance increase caused by that request's claim, or keep a FIFO allocator, and pay only from that amount.

Status

Resolved

[M-02] contracts/delegatecontract/Delegate7702Recovery.sol#recovery - Asset recovery accepts one guardian signature

Description

The verifier is deployed with at least three guardians and a quorum of at least two. Passkey rotation enforces that quorum. `recovery()` accepts one guardian address and one guardian signature and never reads `quorum()`. This is not the documented rule for passkey value movement.

Vulnerability Details

Rotation loads `verifier.quorum()` and checks a guardian set, so one guardian cannot replace the passkey:

```
uint256 required = IDelegate7702Verifier(_guardianVerifier).quorum();
_verifyGuardianQuorum(digest, guardians_, guardianSigs, required);
```

The note on `isValidSignature` says a passkey cannot move value by itself. Value movement through the passkey has to go through `recovery()`, together with the guardian quorum. `recovery()` does not do that. It checks one guardian and one signature, then executes the calls:

```
function recovery(
    Call[] calldata calls,
    uint256 deadline,
    address guardian_,
    WebAuthnProof calldata webAuthnProof,
    bytes calldata guardianSig
) external payable nonReentrant whenDelegated returns (bytes[] memory results) {
    if (!_isGuardian(guardian_)) revert InvalidGuardian();
    address recoveredGuardian = digest.recover(guardianSig);
    if (recoveredGuardian != guardian_) revert InvalidGuardianSignature();
    results = _executeCalls(calls);
}
```

The single-argument signature is how the function is written. The value-movement note still requires the quorum, and rotation is where this codebase enforces `verifier.quorum()`.

Impact

Moving assets through recovery needs the passkey and any one guardian. Therefore, it doesn't need the configured M-of-N guardian quorum.

Recommendation

Accept guardian and signature arrays in `recovery()` and call the same `_verifyGuardianQuorum` check used by passkey rotation.

Status

Resolved

[M-03] Delegate7702Verifier#_changeQuorum - quorum equal to the guardian count is a one-way door, also reached by an ordinary _removeGuardian: no guardian is then removable without unanimity, and one unavailable key makes it permanent

Description

`_changeQuorum` permits `newQuorum` up to and including `guardians.length`, with only a lower-bound check. Once quorum equals `guardians.length`, `_removeGuardian` can never run, because it requires `guardians.length > quorum`. All six ActionTypes then require unanimity, so any single guardian holds a permanent veto over governance, including over its own ejection. The state is reachable two ways, and the second involves no decision about quorum at all.

Vulnerability Details

`_changeQuorum` permits `newQuorum` up to and including `guardians.length`, with only a lower-bound check against `MIN_QUORUM_FOR_SECURITY`. Nothing stops a quorum from raising the threshold to the full size of the set, and the change executes on the current quorum, so an ordinary 2-of-3 majority can walk the set into it. The resulting state looks like hardening.

```
// Delegate7702Verifier.sol - _changeQuorum permits quorum == guardians.length
if (!(newQuorum > 0 && newQuorum <= guardians.length)) revert InvalidQuorum();

// _removeGuardian then becomes unreachable at that quorum
if (!(guardians.length > quorum)) revert CannotReduceBelowQuorum();

// proposeRemoveGuardian refuses first, so the same state is also reached by an
// ordinary removal that happens to land on guardians.length == quorum
if (!(guardians.length - 1 >= quorum)) revert CannotReduceBelowQuorum();
```

The second entry path requires no `CHANGE_QUORUM`. Starting from the deployed 3 guardians at quorum 2, an ordinary `REMOVE_GUARDIAN` for attrition, key rotation or a departure passes both the propose-time and execute-time checks above, and leaves 2 guardians at quorum 2. Quorum now equals the guardian count implicitly. Quorum can never be lowered from there, because the floor pins it at or above

`MIN_QUORUM_FOR_SECURITY` while two or more guardians remain. Routine staff turnover is enough to enter the trap.

The door requires unanimity rather than being sealed. With every key alive and cooperative, a unanimous `ADD_GUARDIAN` restores headroom, taking the set to N+1 while quorum stays at N, after which the target guardian is removable on N-of-(N+1) without its own approval. The state becomes permanent when one key turns hostile, is lost, or stops responding. A compromised guardian is hostile by definition and will never co-sign the `ADD_GUARDIAN` that leads to its own removal, which is the incident-response case.

The team records this state as intentional in the comment above the `MIN_GUARDIANS` constant: "this is a deployment-time floor, not maintained at runtime. Guardian removal only needs to respect the quorum floor (>=2), so the set can shrink to 2-of-2. 2-of-2 loses fault tolerance if one key is lost, but quorum>=2 is maintained so single-key governance takeover is impossible. (Shrinking to 2-of-2 is an intentionally permitted operational state.)" The note rules out a single key forcing an action, which is correct. It does not consider a single key blocking one, which is what quorum equal to the guardian count grants, permanently and unilaterally, across all six ActionTypes.

Impact

At quorum equal to `guardians.length` no guardian is removable and every ActionType needs unanimity, so one hostile, lost or unresponsive key permanently freezes incident response on an immutable contract. A compromised guardian cannot be ejected, a compromised spender cannot be de-registered, and a lost executor cannot be replaced. The largest effect is that `updateWebAuthnKey` reads `quorum` at execution time, so one uncooperative guardian freezes passkey rotation fleet-wide.

Recommendation

Enforce `quorum <= guardians.length - 1` on both mutators: reject `newQuorum >= guardians.length` in `_changeQuorum`, and require headroom after a removal (`guardians.length - 1 > quorum`) in `_removeGuardian`. Fixing only `_changeQuorum` leaves the removal route landing on 2-of-2, where the `MIN_QUORUM_FOR_SECURITY` floor and the invariant cannot both hold, so the `MIN_GUARDIANS` floor of 3 `must` also apply at runtime.

An alternative is to allow removal at `guardians.length == quorum` provided quorum drops atomically in the same proposal.

Status

Resolved

[M-04] Delegate7702Recovery#_executeCalls - A guardian's own delegated account can cast that guardian's governance vote, at a threshold weaker than the guardian's key

Description

`_executeCalls` refuses a call to `address(this)` and to `_implementation`, but places no restriction on `_guardianVerifier`. Under `EIP-7702` the account and the guardian seat are the same address, so a `recovery()` batch run on a guardian's own delegated account reaches `Verifier.approveProposal` with `msg.sender` equal to that guardian. The vote is cast on factors weaker than the guardian's own key. `DELEGATECONTRACT_OVERVIEW.md` raises this as focus area 9 and leaves it open.

Vulnerability Details

`_executeCalls` blocks two destinations and otherwise dispatches with a raw `call`. The Verifier is reachable. approveProposal is gated only by `onlyGuardian`, which tests `_isGuardian[msg.sender]`.

```
// Delegate7702Recovery.sol - _executeCalls blocks only two destinations.
// _guardianVerifier is not one of them.
if (calls[i].to == address(this)) revert SelfCallNotAllowed();
if (calls[i].to == _implementation) revert ImplementationCallNotAllowed();

// dispatched with call, so msg.sender at the target is address(this),
// which under EIP-7702 is the guardian's own address
(bool success, bytes memory returnData) = calls[i].to.call{value:
calls[i].value}(calls[i].data);

// Delegate7702Verifier.sol - which is all that onlyGuardian tests
if (!_isGuardian[msg.sender]) revert NotGuardian();
```

The identity collision is structural rather than forged. Under `EIP-7702` there is no separate wallet contract. The implementation's code executes at the guardian's own address, so `address(this)` is the guardian inside that frame, and every outbound call from `_executeCalls` carries that address as msg.sender. The attacker does not select the identity in the calldata. It follows from which account `recovery()` is run on. Under a

conventional smart-wallet design the batch would execute from a distinct wallet address and `onlyGuardian` would reject it.

The threshold consequence is the substance of the finding. Casting guardian1's vote should require guardian1's ECDSA key. Via this path it requires guardian1's passkey plus any one guardian signature, since `recovery()` places no constraint that `guardian_` differ from the account. For any guardian who has delegated their own EOA and registered a passkey, committee security drops from N guardian keys to one guardian key plus N-1 guardian passkeys.

The stolen authority covers guardian1's whole `onlyGuardian` surface rather than `approveProposal` alone. `revokeApproval` can destroy guardian1's committed approval on an honest proposal, and five proposeAddSpender calls in one batch exhaust guardian1's `MAX_ACTIVE_PER_PROPOSER` budget, after which the real guardian1 cannot propose anything and receives `TooManyActiveProposals`. The capture also outlives its precondition. Once quorum is reached the attacker installs its own guardian addresses and then evicts the honest ones using the real keys it now holds, ending with a guardian set that is entirely attacker-controlled and no longer depends on the stolen passkey.

On likelihood: the design assumes guardians sign from non-delegated keys, but that assumption is not enforced, not documented, and produces no functional symptom. `_addGuardian` checks the zero address, executor and spender overlap, duplication and the cap. There is no code-length or delegation-indicator check, and an add-time check would not be sufficient, since an existing guardian can delegate at any point afterwards. The design also pushes guardians toward being plain EOAs, which is the delegatable class. Every guardian signature path is raw `ECDSA` with no ERC-1271 fallback, since only `ECDSA` is imported, so a Safe or any contract multisig can hold a seat but can never co-sign `recovery()`, `initWebAuthnKey` or `updateWebAuthnKey`. The project's own standard for comparable roles points the other way, since `docs/vault-v2-architecture.md` makes Safe multisig custody mandatory for the vault's privileged roles. That option is unavailable here.

Impact

Casting a guardian's vote should need that guardian's ECDSA key. Here it needs their passkey plus any one guardian signature, dropping the committee from N keys to one key plus N-1 passkeys, and at quorum 2 that is complete governance takeover which becomes permanent on the attacker's own keys. There is a ceiling: captured governance cannot take a wei, because every value path still needs a user-held factor, the EOA key, an owner signature, or the passkey. This is authority capture rather than theft.

Recommendation

Add `_guardianVerifier` to the blocked-target list in `_executeCalls`. It costs one comparison and has no indirect bypass, since the dispatch uses `call` rather than delegatecall. Pair it with a check for the same root cause, which exposes any contract granting privileges by msg.sender: in `recovery()`, revert if `_isGuardian(address(this))`. It must sit in `recovery()` rather than only `initWebAuthnKey`, because guardians can be added later. The EIP-7702 detection primitive already exists in this repository, in `contracts/vault/v2/VaultLib.sol`.

Status

Resolved

[M-05] Delegate7702Verifier#_removeSpender - spender de-registration is a suspension, not a revocation; re-registering revives every authorisation it was meant to kill

Description

`_removeSpender` writes `_isSpenderMap[S] = false` and swap-pops the array. `executeBatchCallsWithSig` gates only on current spender membership and on the owner digest. Nothing binds an authorisation to a registration epoch, so the mapping flip is reversible and the authorisations survive it. Three NatSpec sites document de-registration as the bulk kill-switch for a leaked spender key, and the code does not deliver it.

Vulnerability Details

`_removeSpender` writes `_isSpenderMap[S] = false` and swap-pops the array. `executeBatchCallsWithSig` gates only on `_isSpender(msg.sender)` and on the owner digest. `BATCH_EXECUTION_TYPEHASH` carries four `fields`, `callsHash`, `spenderBatchNonce`, `deadline` and `spender`, and no registration epoch, so the mapping flip is reversible and the authorisations survive it. The nonce advances only on successful execution, so de-registration moves nothing. The four places that document it as a bulk kill-switch are the NatSpec on `executeBatchCallsWithSig`, the comment above the `_isSpenderMap` declaration, the `@dev` block on `proposeRemoveSpender`, and the double-gate note in `DELEGATECONTRACT_OVERVIEW.md`.

```
// Delegate7702Verifier.sol - de-registration is one flag, and it is reversible
_isSpenderMap[spenderToRemove] = false;

// Delegate7702Recovery.sol - execution tests only current membership
if (!_isSpender(msg.sender)) revert InvalidSpender();

// and the digest carries no registration epoch, so a surviving signature
// becomes valid again the moment the address is re-added
keccak256(abi.encode(BATCH_EXECUTION_TYPEHASH, _hashCalls(calls),
s.spenderBatchNonce, deadline, msg.sender))
```

The root cause is structural rather than a missing field. There is no fleet-wide invalidation primitive, and there cannot be one in the current layout. `spenderBatchNonce` lives in per-account storage, while the spender registry lives in the global Verifier, which cannot reach into any account's storage. Governance's only global lever over outstanding authorisations is the membership flag, and that flag is reversible by construction. After governance does everything available to it, the account's spenderBatchNonce is unchanged.

The mechanism needed for a fix already exists in the Verifier and is not consulted. `_removeSpender` bumps `policyVersion`, which `_isProposalStale` uses to retire in-flight proposals on every role mutation. `executeBatchCallsWithSig` never reads it. Across two suspend-and-restore cycles, policyVersion advanced four times while the account-layer authorisation stayed valid.

Two conditions shape the real exposure. The nonce is burned by ordinary traffic, so a single legitimate spender batch permanently kills any stale authorisation sharing that nonce value, and active accounts largely protect themselves. Exposure concentrates on dormant accounts, and on queued or pre-signed authorisations. In the other direction there is no maximum deadline, so the revival window is not bounded by any incident-response timeframe. A 10-year deadline is accepted, and an authorisation was observed executing 8 years after suspension.

Impact

Four places document de-registration as the bulk kill-switch for a leaked spender key, but it clears `_isSpenderMap[S]` only. Authorisations keep their nonces and deadlines, so re-registering the address revives every one. The trigger is routine rather than a blunder, since restoring a spender after a false-positive alert is correct. Remediation is per-user, not per-incident: `invalidateSpenderBatchNonce` needs the owner's own key, and a key-lost account can never invalidate. Loss is bounded by what the owner already signed, and the finding is rated Medium, held there by the documentation contradiction.

Recommendation

Bind authorisations to a registration epoch: a per-spender counter incremented on both add and remove, folded into `BATCH_EXECUTION_TYPEHASH`, so de-registration permanently retires that generation. The counter must be per-spender rather than the global `policyVersion`, which all six ActionTypes bump and which would invalidate every outstanding batch fleet-wide on any unrelated role change. Capping `deadline` shrinks the window on its own. If the epoch fix is deferred, correct all four documentation sites first, since runbooks get written against a kill-switch claim that does not hold.

Status

Resolved

Low

[L-01] contracts/vault/v2/VaultV2.sol#simulateBuffer - Buffer simulation omits the Prime funding reserve

Description

`simulateBuffer()` is documented as a simulation of `expectedLiquidity`. The live calculation includes the Prime funding reserve. The simulation does not, so an off-chain backtest can understate the liquidity the vault actually requires.

Vulnerability Details

Live `expectedLiquidity` adds the withdrawal-queue totals and then the Prime funding reserve. When Prime is inactive that reserve is zero:

```
amount = (adjusted * _paramsStorage().bufferMultiplier) / 10_000;
amount += _aggregateQueueLiquidity();
amount += _libFundingLockupReserve();
```

`_libSimulateBuffer`, which `simulateBuffer` returns, stops before that last term:

```
amount = (adjusted * _paramsStorage().bufferMultiplier) / 10_000;
amount += _aggregateQueueLiquidity();
```

When no Prime queue is set, the missing term is 0 and the two results match. When Prime funding reservations are active, `fundingLockupReserve()` is a positive urgency-weighted sum, and the view used for backtests is short by that amount. The function is `view` and currently doesn't move funds.

Impact

An operator or backtest that trusts `simulateBuffer` can treat the vault as having enough liquid buffer when live `expectedLiquidity` is higher by the current Prime funding reserve.

Recommendation

Add `_libFundingLockupReserve()` in `_libSimulateBuffer` after the queue total, using the same behavior as `_libExpectedLiquidity`.

Status

Resolved

[L-02] Delegate7702Recovery#invalidateSpenderBatchNonce - all three +1 revocation levers promote the next pre-signed authorisation instead of destroying it

Description

All three emergency revocation levers add 1 to a counter that the corresponding digest pins by equality, so an authorisation signed at counter+1 is dormant rather than void. One invalidation moves it from unsubmittable to submittable. The NatSpec on each of them promises the call invalidates every signed but unsubmitted approval, and against a pre-signed queue it does the opposite for the next entry.

Vulnerability Details

The defect affects `invalidateSpenderBatchNonce`, `invalidateRecoveryNonce` and `invalidateWebAuthnUpdateNonce` identically, so it is one defect in three places. Each is the documented escape hatch for its signing surface, which puts the defect on every emergency path the account has.

```
// Delegate7702Recovery.sol - one defect in three places. Each is a step, so an
// authorisation signed at counter+1 is promoted rather than killed.
s.spenderBatchNonce += 1;   // invalidateSpenderBatchNonce
s.nonce += 1;               // invalidateRecoveryNonce
s.webAuthnUpdateNonce += 1; // invalidateWebAuthnUpdateNonce
```

There is also no way to clear a queue in one action. Each lever moves its counter by exactly one, and `_executeCalls` forbids batching self-calls, so retiring several pending authorisations takes that many separate mined transactions. Each gap between them is a window a block builder can order a submission into, and every step promotes one more queued entry on the way through.

Impact

All three levers add 1 to a counter that the batch, recovery and rotation digests each pin by equality, so an authorisation signed at counter+1 is dormant rather than void, and one invalidation makes it submittable. All three NatSpec blocks promise invalidation of every signed but unsubmitted approval, and against a queue each call does the

opposite for the next entry. The owner cannot burn several at once either, since `_executeCalls` forbids batching self-calls. Against an attacker holding a prepared queue the lever is counterproductive, because it advances the counter into the queue.

Recommendation

Make invalidation a jump rather than a step: set the counter to a caller-supplied value that must exceed the current one, or keep a separate `minAcceptedNonce` that the digest must meet or exceed. Either approach kills an entire queue in one transaction.

Status

Acknowledged

Centralisation

The Line Next project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the Line Next project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


hashlock.